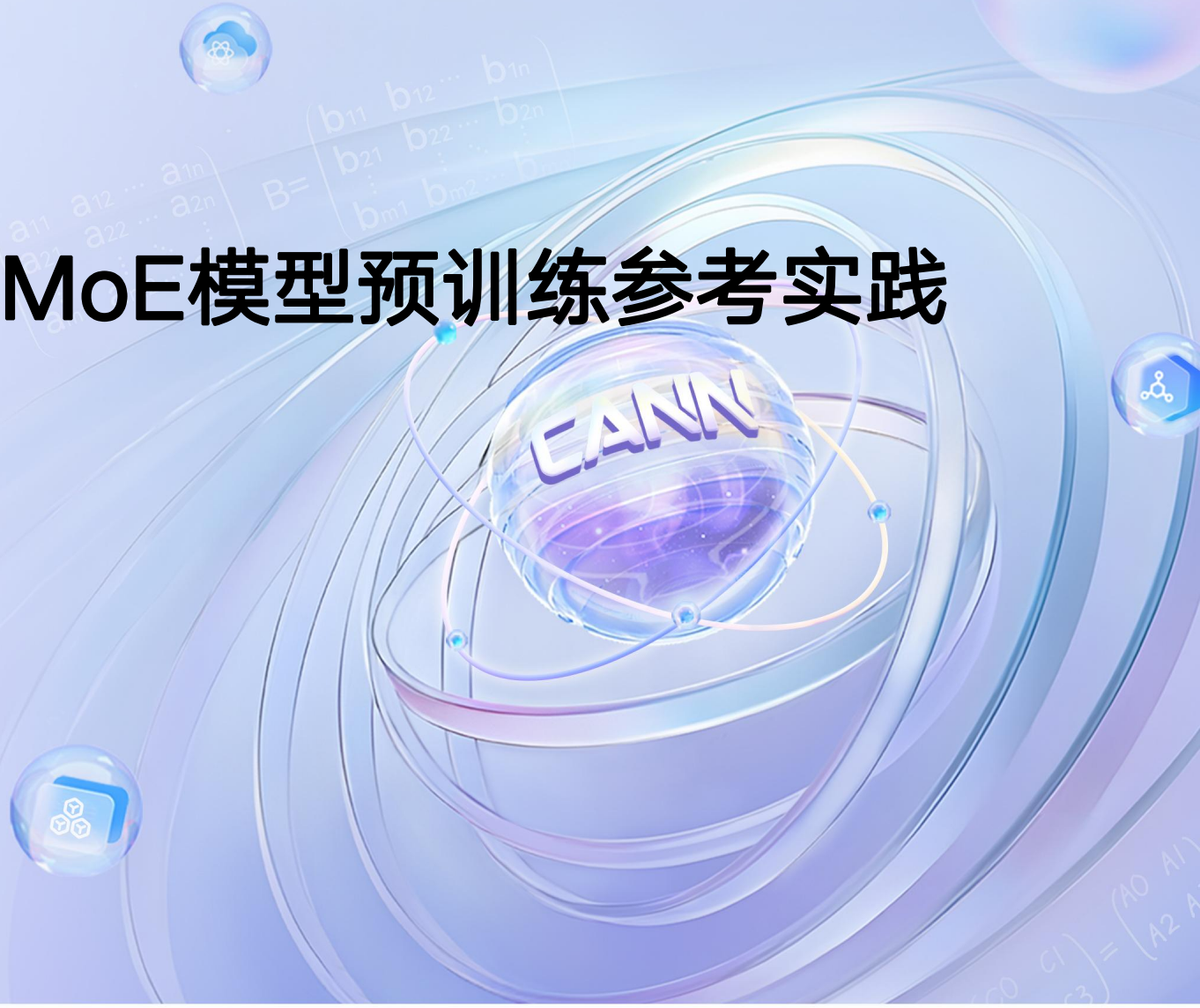
基于950超节点的万亿MoE模型预训练参考实践

讲师：潘超、杨喆军、郭悦



CANN

目录

基于950超节点的万亿MoE模型预训练参考实践

Part 1：模型训得动
- Pytorch原生训练框架下的950超节点部署策略
- 支持Tilelang算子接入、 Ascend C 算子的自动融合

Part 2：模型训得好
- 950全局统一虚拟地址：整网内存优化
- 128p 950实现1M长序列部署能力
- 训练图优化能力
- 高效分布式muon优化器

Part 3：训练高可用
- 面向千卡、万卡集群的预训练高可用能力

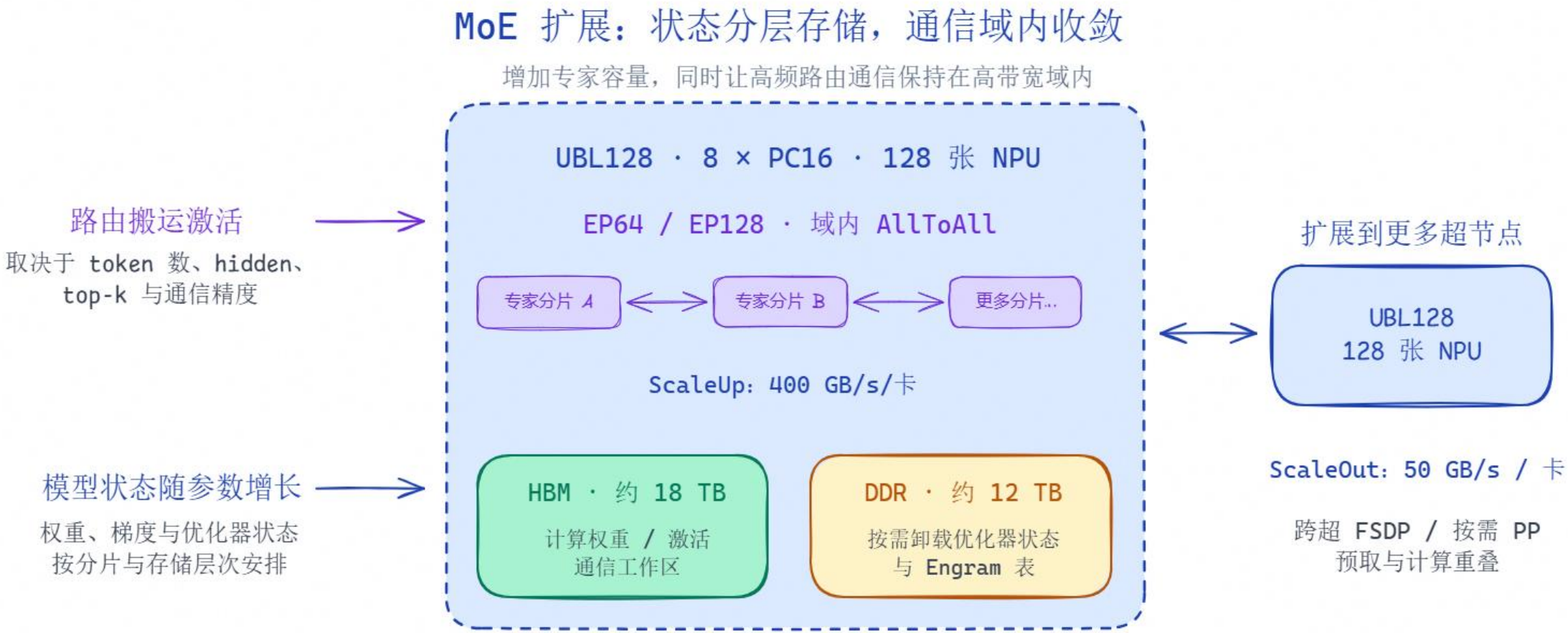
目录

Part 1 模型训得动

Part 2 模型训得好

Part 3 训练高可用

1. Pytorch原生训练框架下的950超节点部署策略



1. 950超节点部署策略：模型部署策略对比

对比维度	550B：DeepSeek V4.1 Flash	1.6T：DeepSeek V4 Pro	10T：规模外推
代表性切分	Dense部分 FSDP128，EP64 + 专家侧 eFSDP2；EP128 可作容量与负载对比候选	MoE EP128 × FSDP2；Attn 在 CP1 – 256、FSDP1 – 256 范围内选型	EP128 × FSDP4 × PP2 为拓扑候选，FSDP 与 PP 根据模型调整
单副本资源	1 × UBL128，128 张 NPU	2 × UBL128，256 张 NPU	上述 PP2 候选为 8 × UBL128、1024 张 NPU
EP 通信范围	64 或 128 卡组，均在超节点内	每个 UBL128 内的 128 卡组	每个 UBL128 内的 128 卡组，随扩容保持不变
主要扩容手段	域内 FSDP，按需卸载 Engram 与优化器状态	跨两个超节点做 FSDP2，不切 PP	跨超节点 FSDP 分摊状态，并通过 PP 分摊层与临时聚合权重
获得的收益	超节点内带宽支持权重分片、高速通信	无 PP 气泡；所给单层方案中，FSDP 跨超通信量与耗时均小于 PP 切分	控制每个 stage 的模型状态和临时聚合权重，保持 EP 通信局部性

- PC16 集群上 DeepSeek-V4-Flash **64 卡** 4K 序列训练吞吐达到 **2131 tokens/s/卡**

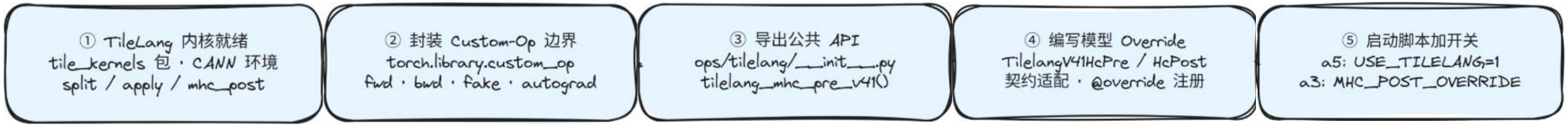
2. 支持Tilelang算子接入、支撑训练算法快速迭代

Tilelang算子接入：让高性能算子开发更简单！

- 分层且可混合的编程模型
- 面向昇腾存储层级的自动映射
- 依赖感知的编译优化
- 并行与流水抽象
- 从开发到调优的工具闭环

基于 DeepSeek-V4 / DeepSeek-V4.1 网络在昇腾 950 系列完成了对 TileLang 的支持。





https://gitcode.com/cann

训得动

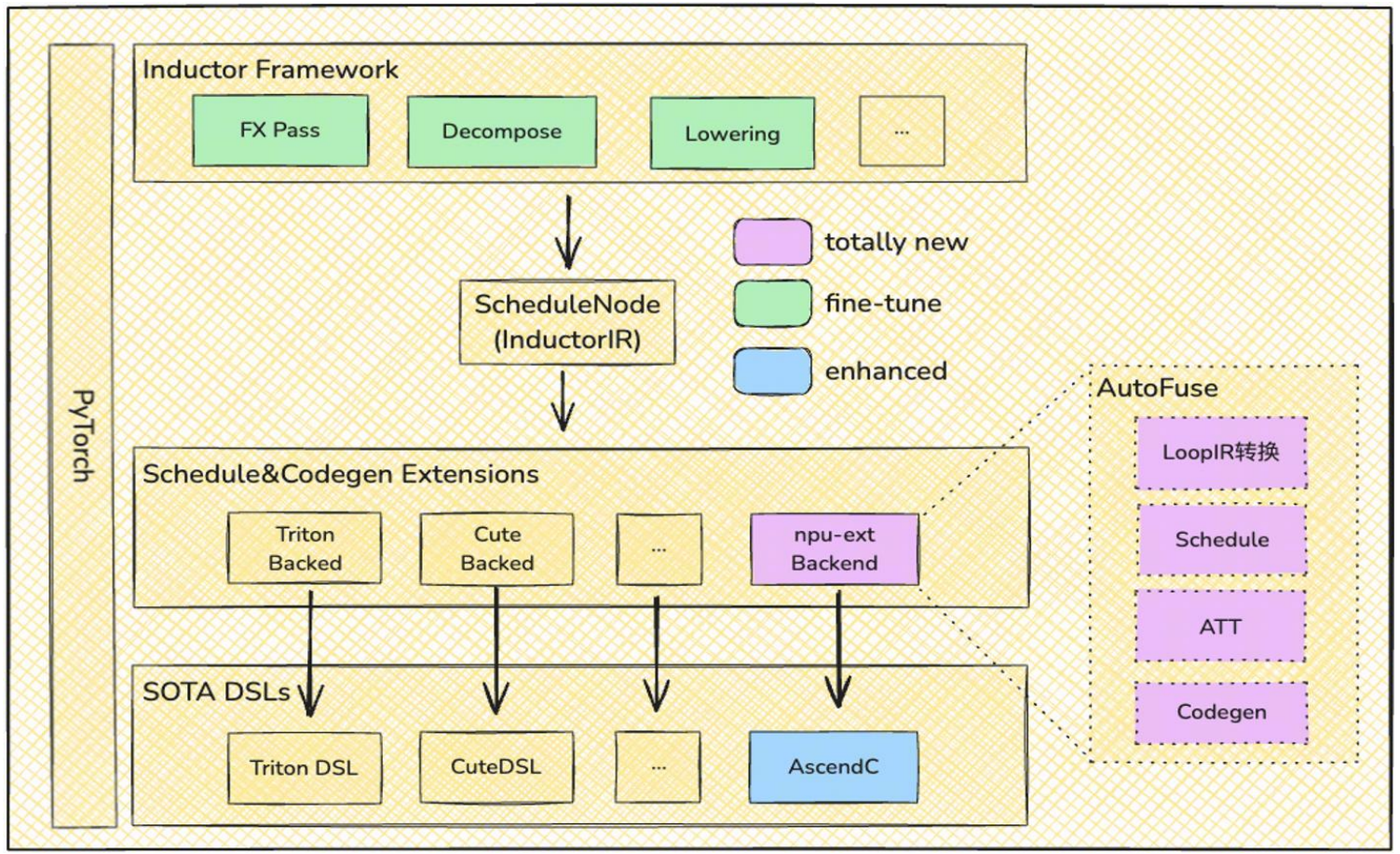
950超节点部署策略	Tilelang与自动融合	整网内存优化	1M长序列训练	训练图优化能力	高效分布式muon优化器	高可用能力

2. 支持 Ascend C 算子的自动融合，加速训练算法的迭代更新

使用简单：PyTorch 前端增加一行代码即可使能

- ✓ 亲和 NPU，模板规约的 Schedule 技术
- ✓ 基于硬件建模，动态求解的算子 Tiling 技术
- ✓ 基于 Ascend LoopIR 表达的，Ascend C 算子 kernel 代码生成技术

Torch.compile inductor + AutoFuse



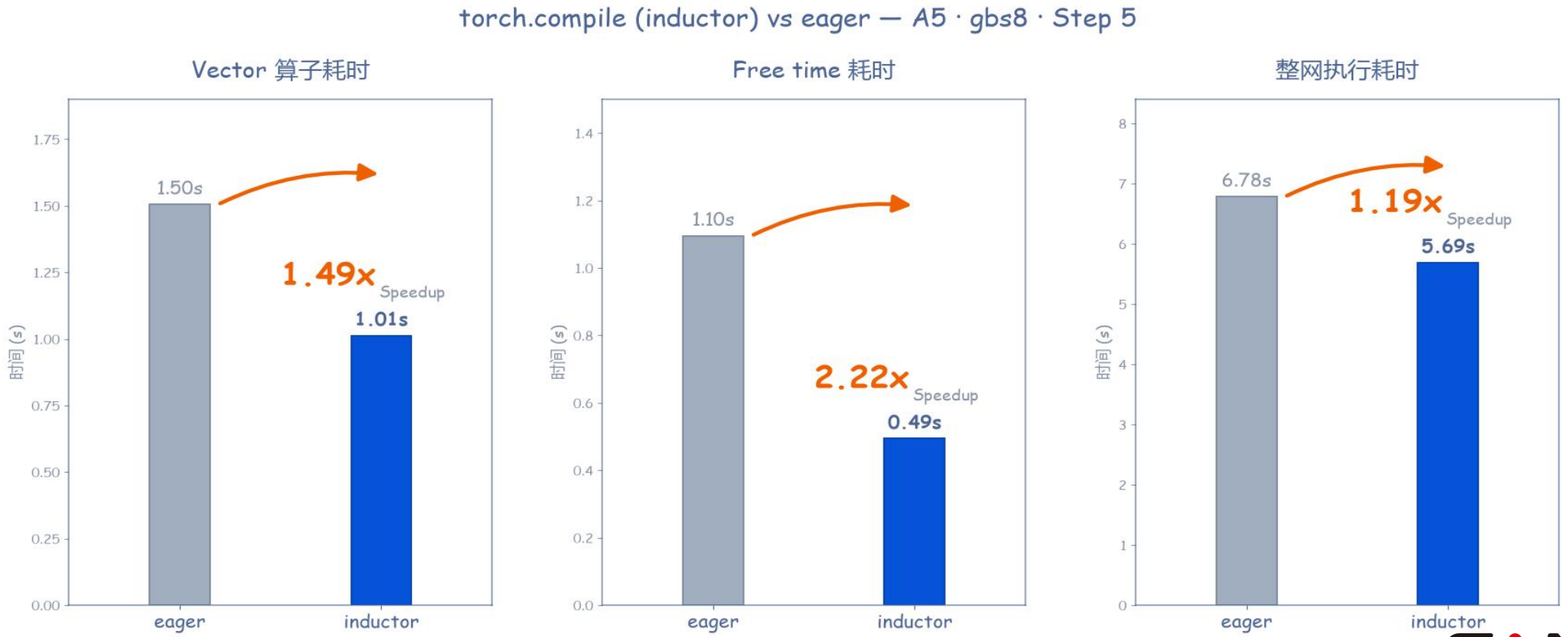
https://gitcode.com/cann

CANN

2. 支持 Ascend C 算子的自动融合：使能收益

使能 AutoFuse 后 DeepSeek-V4.1-Flash 裁剪模型整网吞吐收益为19%

- 消除“访存墙”
- 掩盖高耗时指令
- 精简内核启动开销



https://gitcode.com/cann

CANN

目录

Part 1 模型训得动

Part 2 模型训得好

Part 3 训练高可用

3. 950全局统一虚拟地址：整网内存优化（细粒度checkpoint、全Offloading）

细粒度重计算	CPU OFFLOAD	Engram表
• 基于GraphTrainer，在 FX 图中指定参与重计算的节点	• 参数、梯度和动量结合前向、反向及优化器调度，支持卸载到 CPU • 计算掩盖传输	• 按需访问表行 • MXFP8 查表数据，数据传输体积减少到约为 FP32 的 25.8% • FP32 主表由 CPU SparseAdam 更新命中行

DeepSeek-V4.1-Flash 550B 理论上需要在 128P 上才能完成训练部署，当前**基于整网内存优化方案**，将梯度和参数以及engram表卸载到CPU上，dsv4.1 550B 在可以在仅**64p**昇腾950上跑通训练

实验配置	卡数	Memory	tps
不开启offload	64	200.79GiB（静态内存130.79GiB，激活内存按70GiB估算）	/
开启参数+梯度 offload	64	136.49GiB（静态内存66.49GiB，激活内存按70GiB估算）	/
开启参数+梯度+engram offload	64	71.79GiB	440

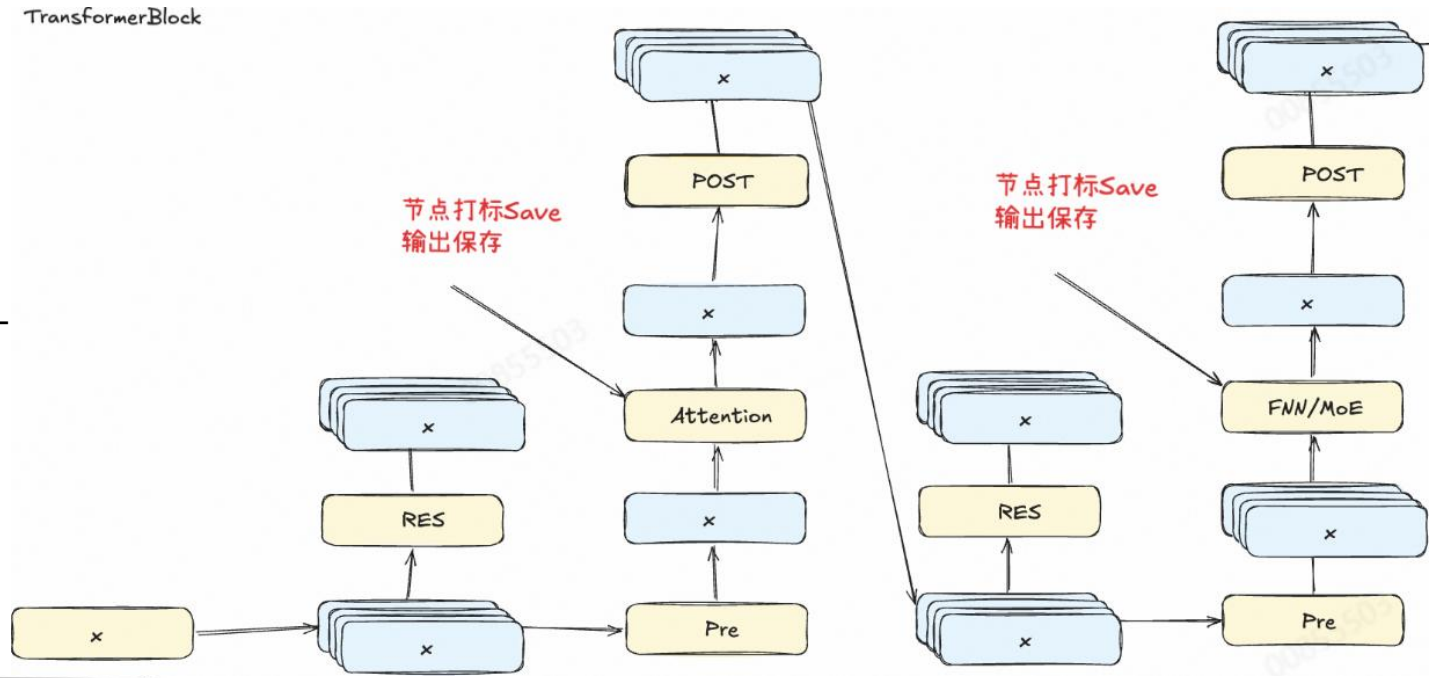
https://gitcode.com/cann

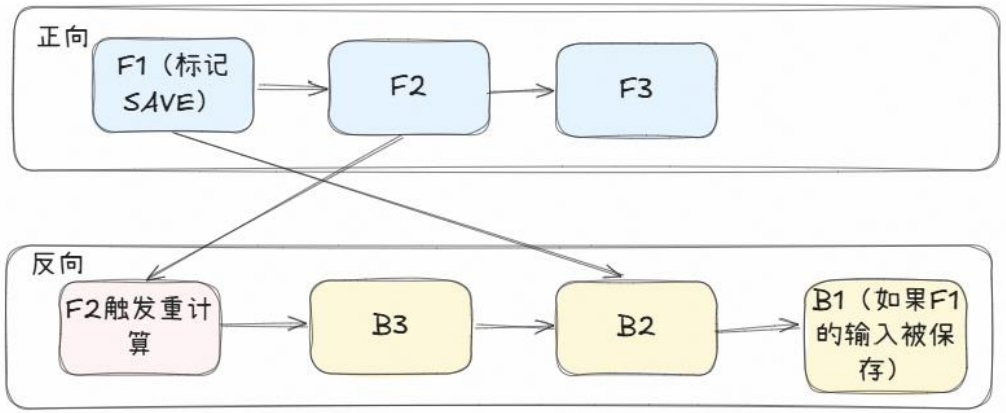
CANN

3. 整网内存优化：细粒度checkpoint

GraphTrainer：从 Op/FQN 粒度扩展到精准 Node 粒度

- Node 级重计算
- 精准定位：以 FQN + Node Target + Occurrence 唯一定位目标 Node，并支持跨层输出保存
- 依赖感知：自动跳过无反向依赖的计算/通信





mHC策略：面向关键 Tensor 的分段重计算

- 保存关键 Tensor：Attention 模块输出、MoE 模块输出
- 重计算流程：
  保存 Attention/MoE 输出→ 前段 BWD → 释放已使用激活 → 后段重计算+激活保存 → 后段 BWD

https://gitcode.com/cann

3. 整网内存优化：CPU-OFFLOAD

前向计算

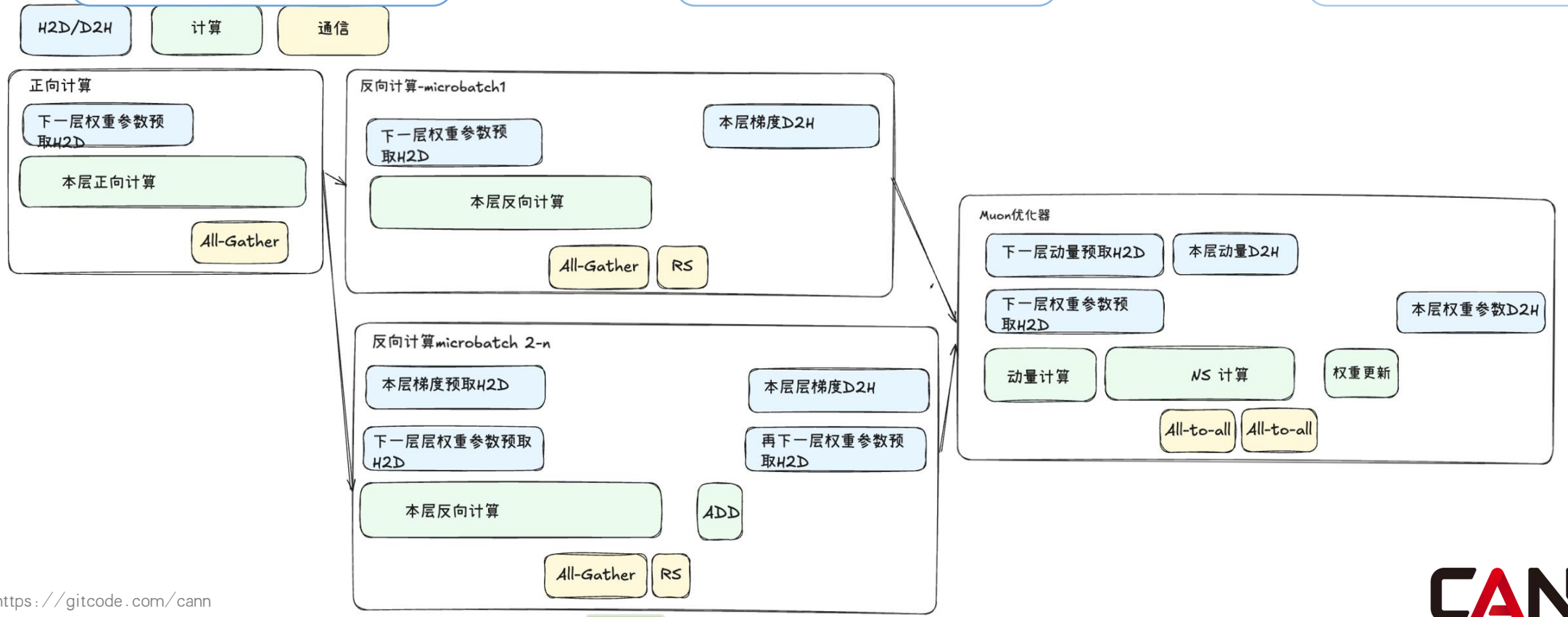
- 参数按层预取
- FSDP 协同
- 非分片 experts 特例

反向计算

- 两条传输流并发执行
- 梯度累计特殊处理

优化器

- 梯度裁剪后驻留 NPU：省去一次 D2H→H2D 往返
- Muon优化器中权重、动量的 swap 按层预取

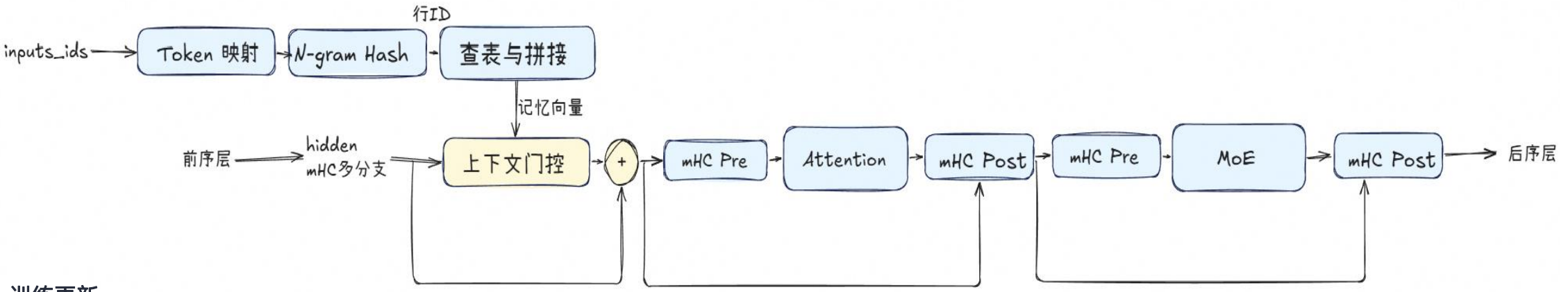


https://gitcode.com/cann

CANN

训得好

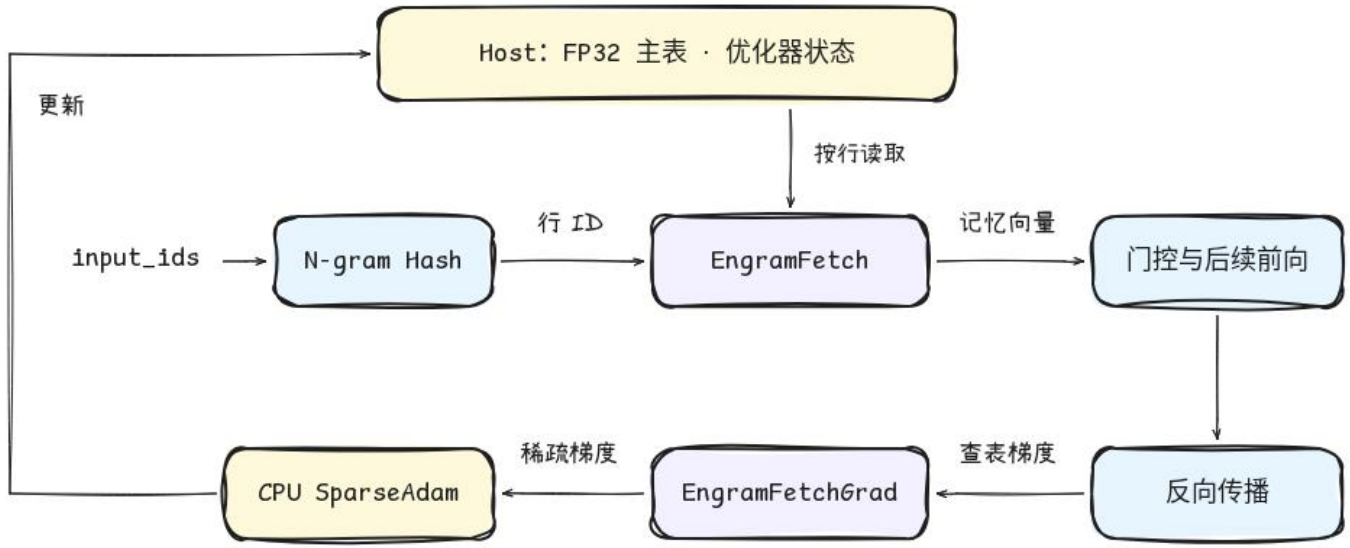
3. 整网内存优化：Engram



训练更新

Engram主表和优化器状态常驻 Host（CPU 内存），按以下流程完成一次**训练更新**：

前向查表 → **反向归并** → **权重更新**



AscendC 训练路径:

- 存储注册：engram_write 将稳定的锁页 Host 表注册为通信可达的存储。
- EngramFetch：接收全局行 ID，由所属 rank 读取本地表后返回数据，并恢复原请求顺序。
- EngramFetchGrad：将梯度按本地行归并，返回行 ID 与梯度。

CANN

4. 仅128p 950实现1M长序列部署能力

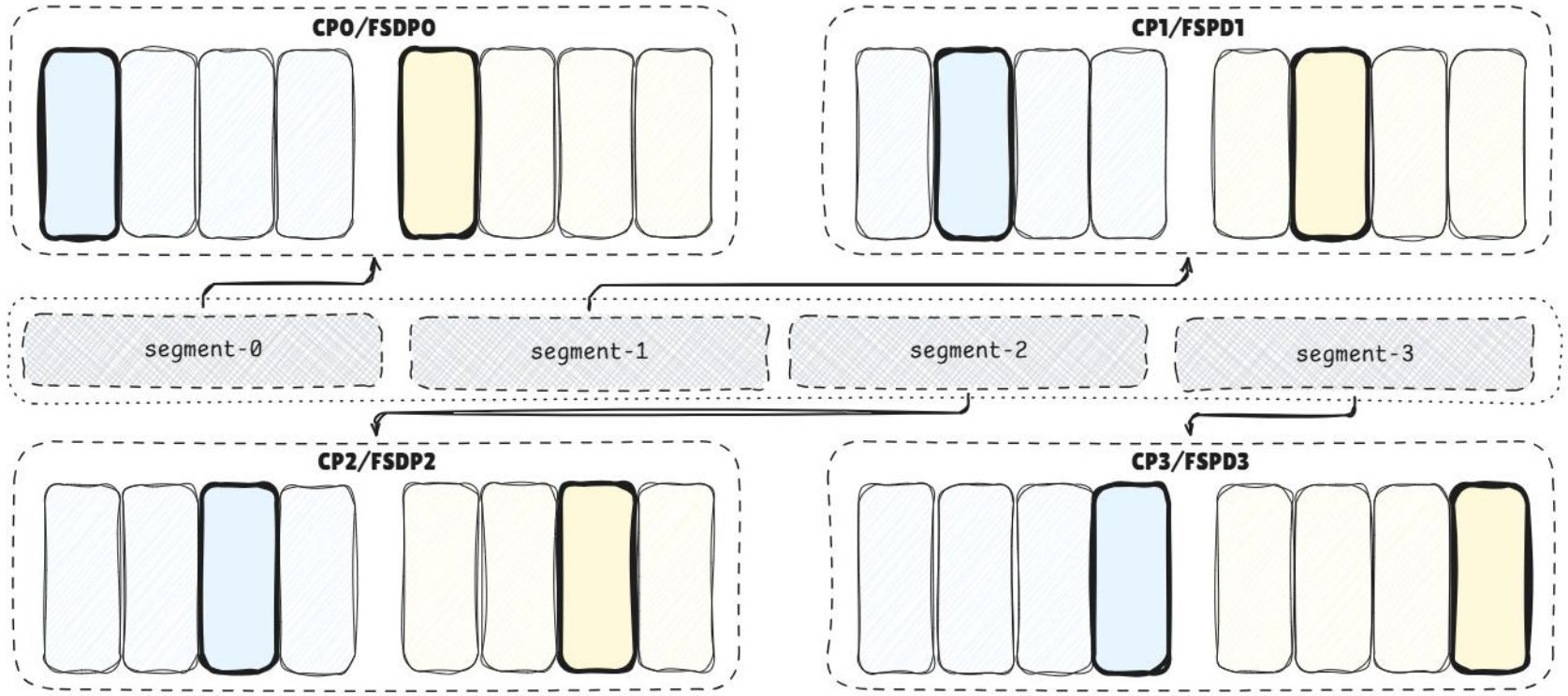
背景：

- FSDP：每张卡在逻辑上部署完整的模型权重，按照DP域的模式处理
- 序列切分：切分的是输入的激活值

因此FSDP与CP可以完美共存

KVAllGather：

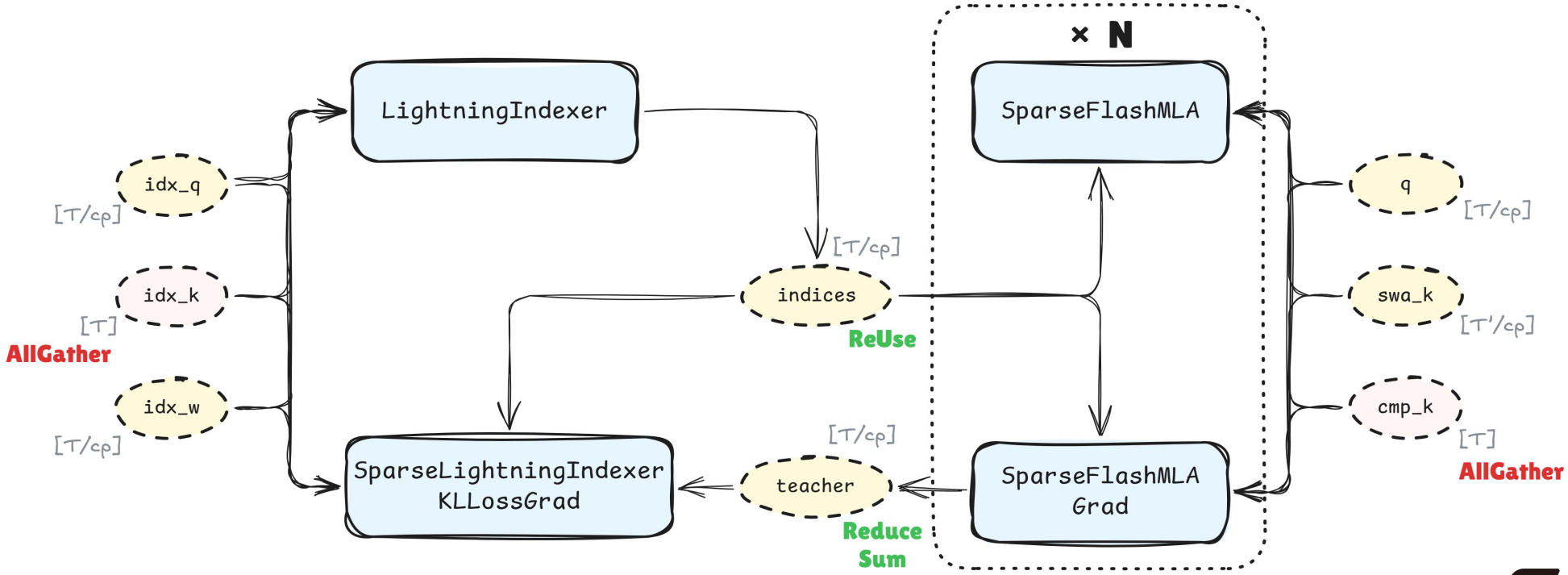
- KV维度天然精简
- Indexer融合算子的兼容性



4. 仅128p 950实现1M长序列部署能力：融合算子实现

融合算子实现

LI	SMLA	SMLAG	SLIKG
• 对完整KV打分并输出Top-K索引	• 在稀疏选中的KV上执行FlashAttention前向	• 对应的反向梯度算子	• Indexer反向算子

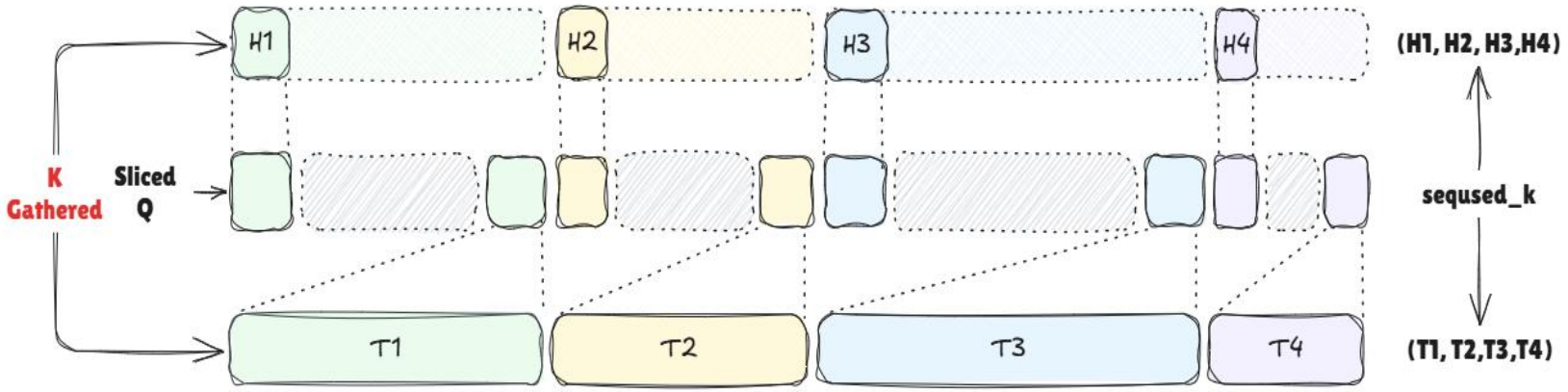


https://gitcode.com/cann

CANN

训得好

4. 仅128p 950实现1M长序列部署能力：整网优化



KVAllGather序列并行与ZigZag负载均衡实现

- 序列压缩块的边界对齐
- 序列顺序保持与低秩Q重排
- Varlen场景下的负载均衡

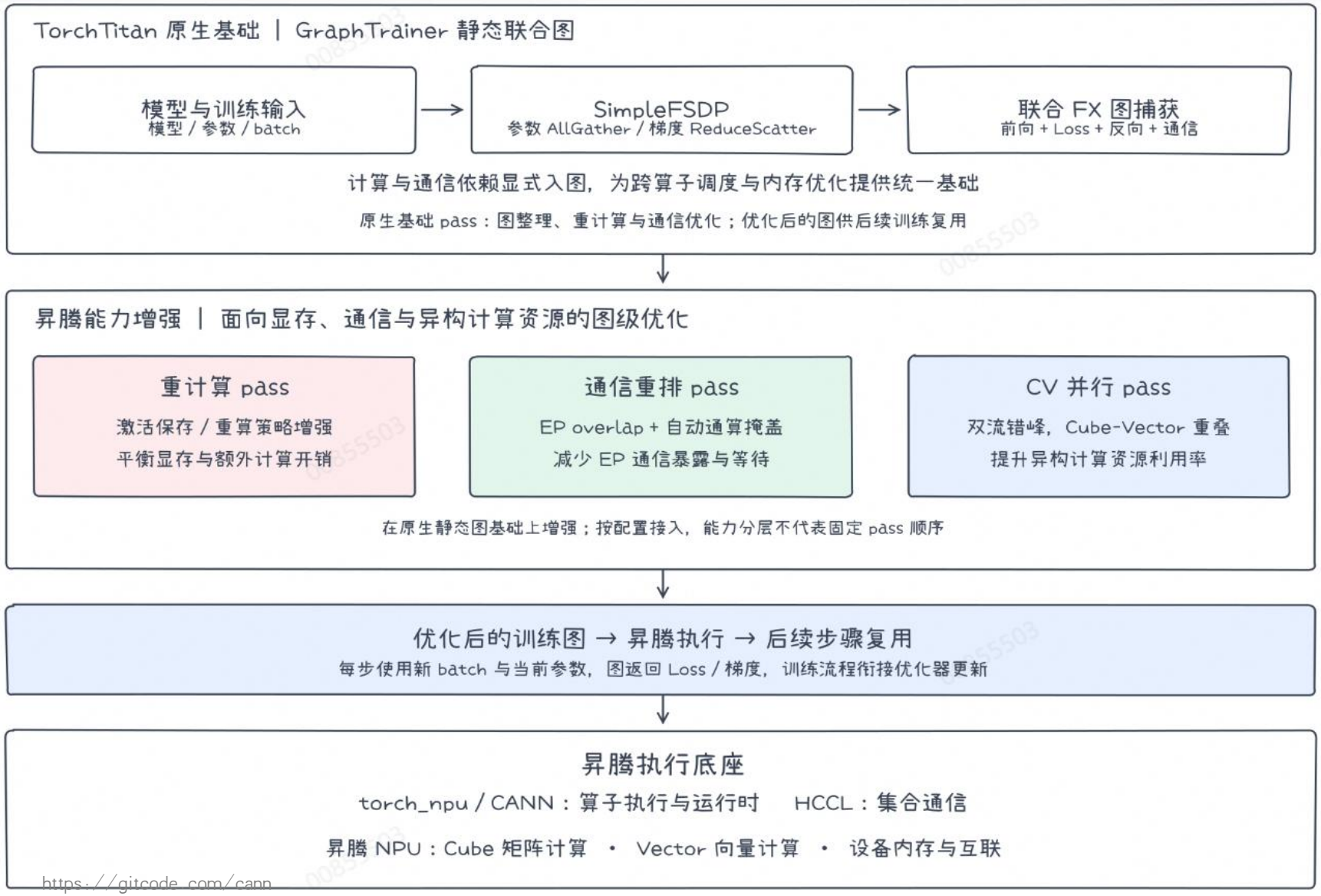
1M长序列训练实测

基于KVAllGather序列切分方案，我们已在128P昇腾950超节点上实际实现了DeepSeek-V4系列模型的1M长序列训练，相对于非负载均衡场景，实测数据集上的未掩盖通信占比**由56.95%降低至39.3%**。

配置	内存占用（GB）	吞吐（tokens/s/卡）
128P · 1M	77.95	1441

分解项	128P · 1M 耗时（s）	耗时占比
计算合计	26.220	57.7%
未掩盖通信合计	17.890	39.3%
FREE（流水空隙）	1.369	3.0%
E2E	45.479	100.0%

5. 训练图优化能力：Profile Aware通算/CV掩盖



整网收益

在 950 双机、序列长度 4096、MBS=2 的 DeepSeek-V4 裁剪配置下，

- GraphTrainer BF16 的实际吞吐由 FSDP2 eager 的 1696 TPS 提升至 1913 TPS，提升 **12.79%**，
- 通信未掩盖时间从746ms优化到347ms，占比 **3.84%**。

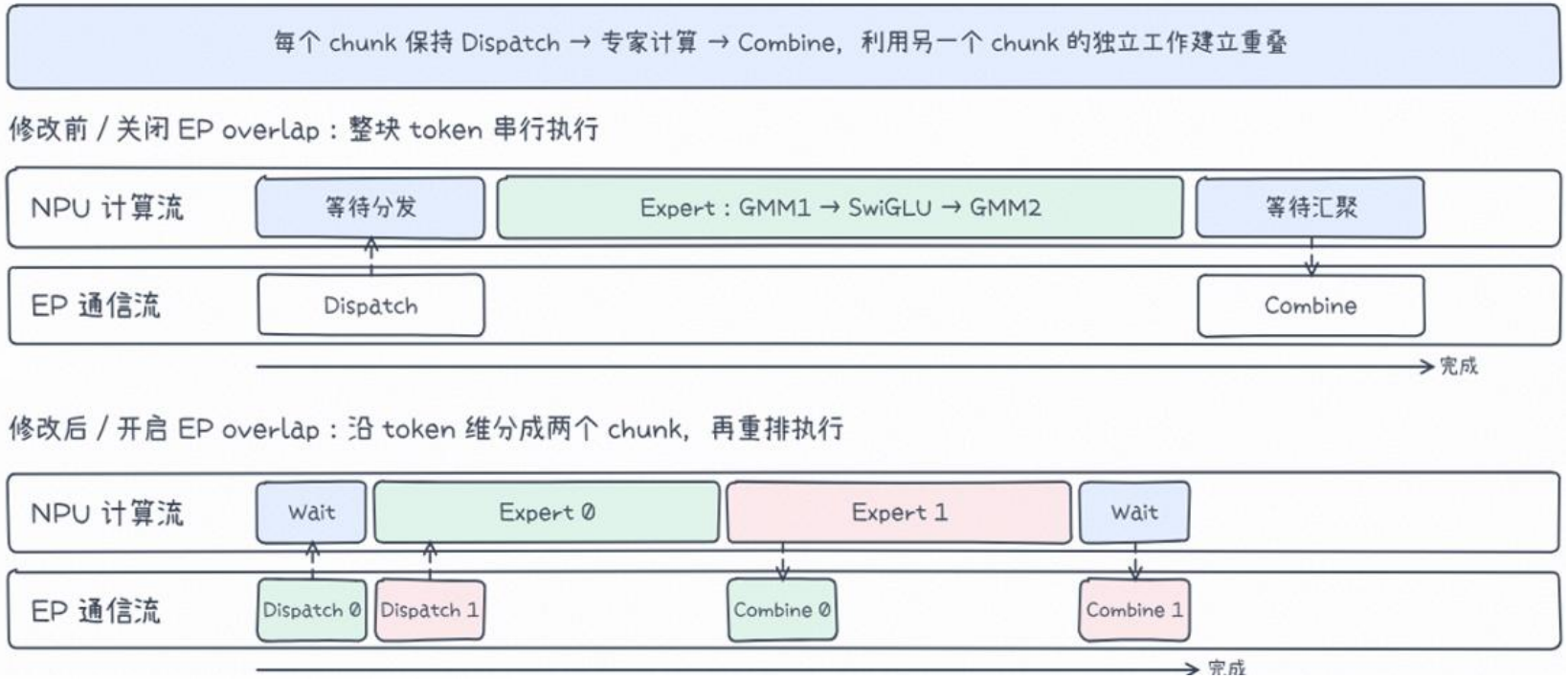
CANN

训得好

5. 训练图优化能力： EP overlap

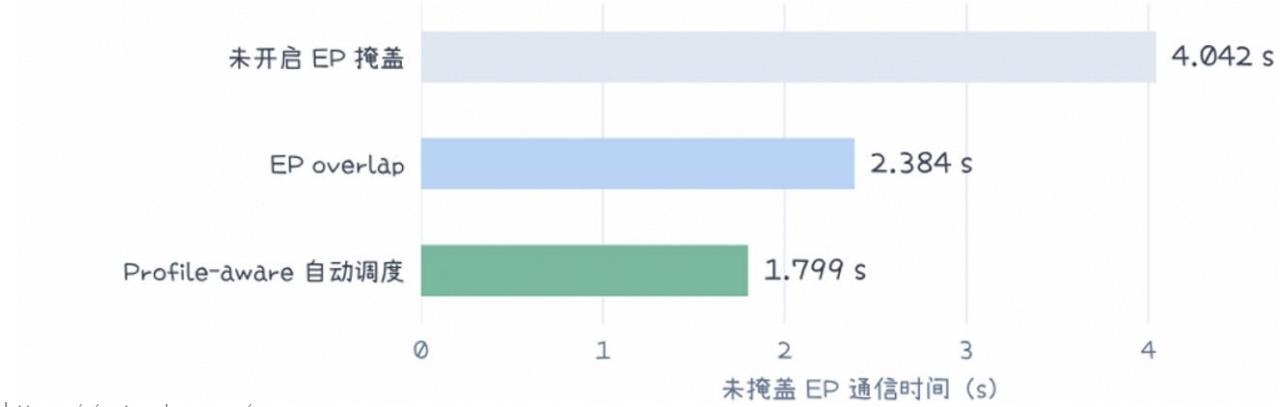
EP overlap：

基于 HyperParallel 的静态图技术，结合 TorchTitan 社区的双 chunk EP 切图，在 GraphTrainer 中推进 EP 通信掩盖：先形成可重叠的 MoE 图，再利用静态图依赖与设备 Profile，按实测通算成本决定节点顺序。



未掩盖 EP 通信减少 55.49%（2.243 s）

自动调度较固定 EP overlap 再减少 24.54%（0.585 s）



收益

未开启 EP 掩盖时，baseline 单步时间约 **40.14 s**，其中未掩盖 EP 通信为 **4.042 s，占 10.07%**。

https://gitcode.com/cann

CANN

训得好

5. 训练图优化能力：Profile Aware通算掩盖

单算子 Profile	按耗时调度	整图 Profile
• 在目标 NPU 上分别执行计算与通信算子的 benchmark，测量设备侧耗时，形成节点级 cost model。	• 调度器根据 cost model 估计通信剩余时间和下一次通信的输入准备耗时，优先保持通信连续下发，再用已就绪的独立计算填充窗口	• 执行上一轮排好的完整 GraphTrainer 图，采集计算 kernel、copy 和 HCCL 通信在真实执行上下文中的耗时，关联回 FX 节点并进行跨 rank 对齐。



https://gitcode.com/cann

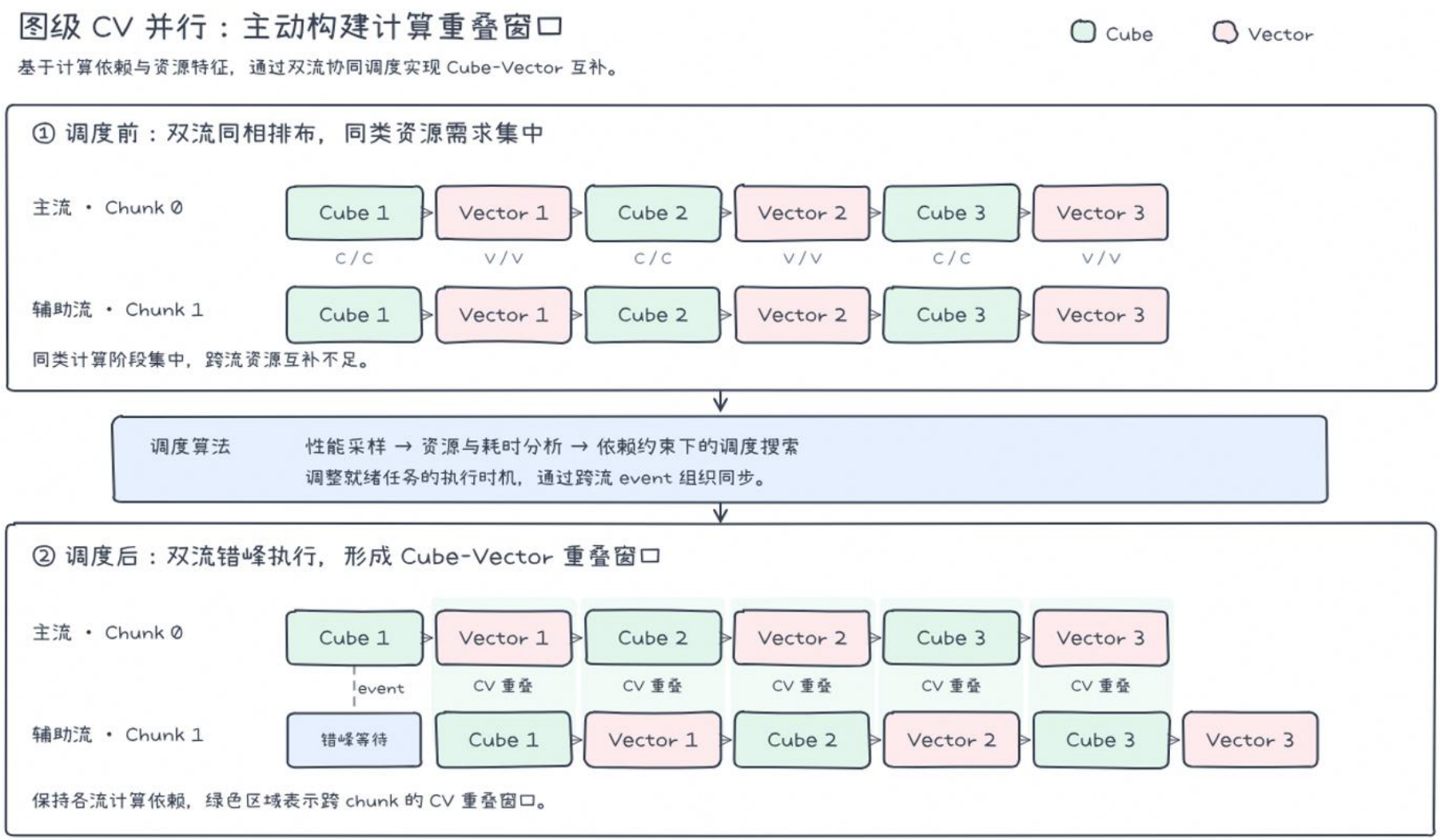
5. 训练图优化能力：CV掩盖

CV重叠

图级 Cube－Vector（CV）并行沿 batch 维度拆分两个 chunk，并将它们安排在两条计算流上。

收益

从各类资源自身的执行时间看，**31.959%** 的 Cube 时间与 Vector 重叠，**34.645%** 的 Vector 时间与 Cube 重叠；在本次配置下，计算并集同时缩短了 **376.391 ms**。



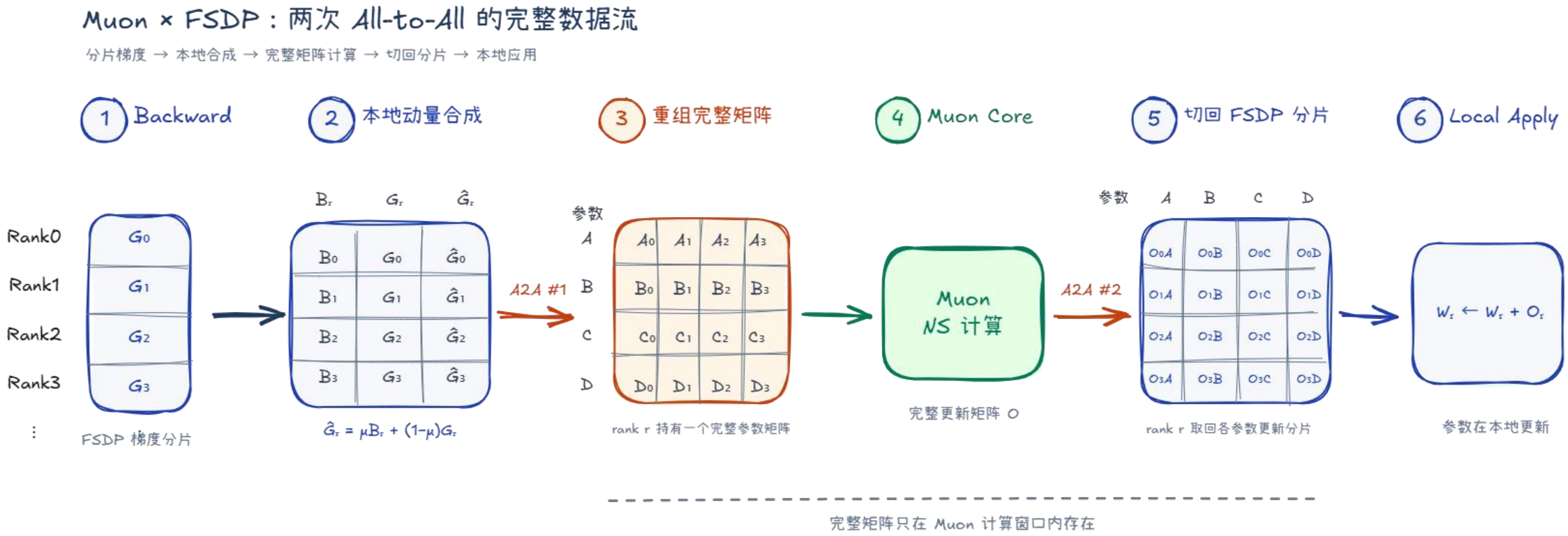
https://gitcode.com/cann

CANN

6. 高效分布式muon优化器

高效分布式muon优化器接入：

- FlexMuon 通过按需参数布局重组与层级流水，重叠 AllToAllV 通信、Host－Device 状态搬运和 Newton－Schulz 计算，并以 NovaSwap 解耦优化器状态管理。结合 HSDP 的“计算一次、广播结果”和昇腾 950 Cube－Vector 融合 Kernel的计算优化，减少重复计算、常驻显存与中间数据搬运。



https://gitcode.com/cann

CANN

6. 高效分布式muon优化器：计算、通信、内存协同优化

通信优化

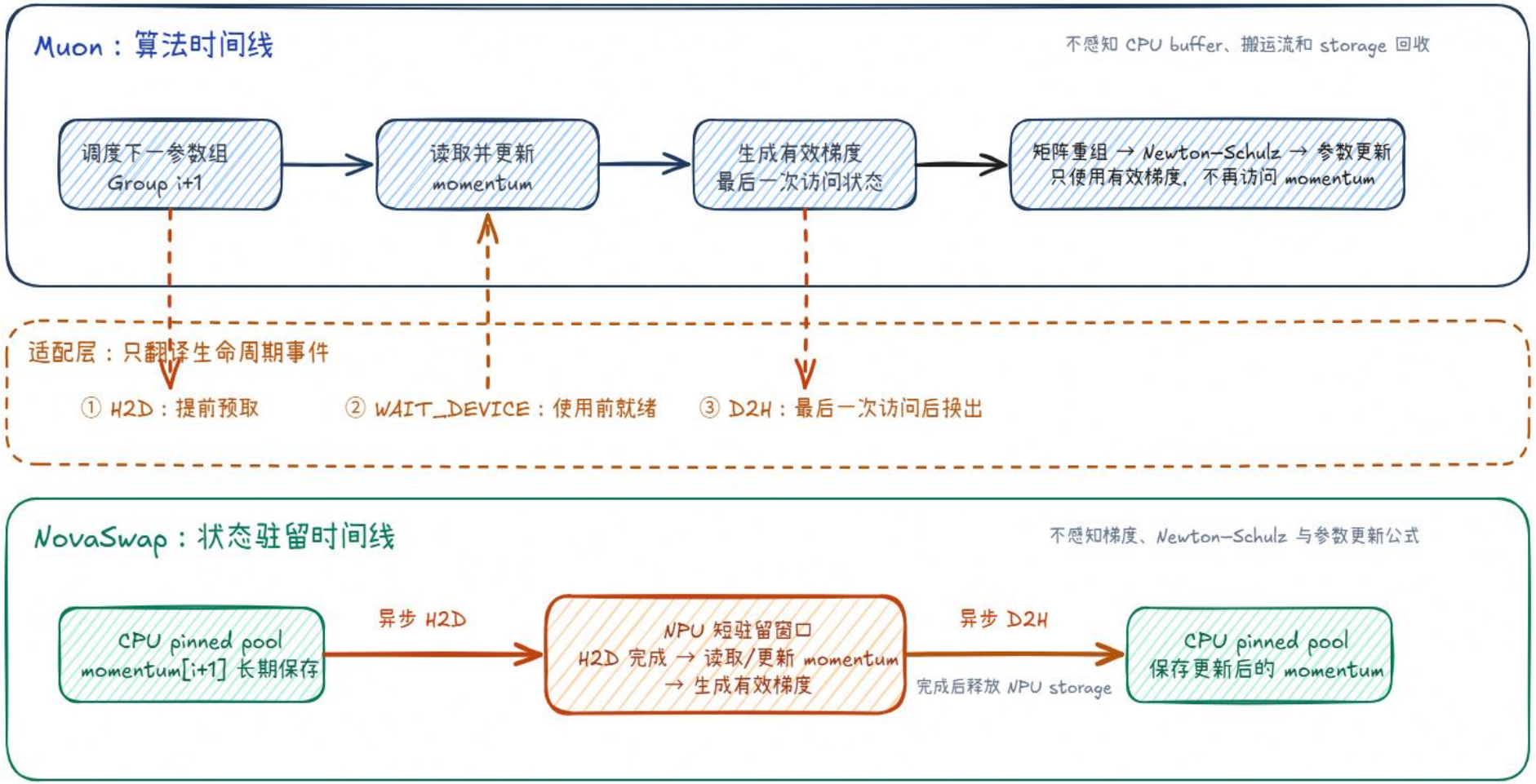
- 普通 Dense 权重：两次 AllToAllV 完成布局转换
- 本地完整矩阵：按 batch 执行 Newton－Schulz

内存优化

- 按参数组换入、计算与换出。
- CPU 内存池承接长期状态

计算优化

- 使用昇腾 950 Cube－Vector 融合 Kernel



https://gitcode.com/cann

CANN

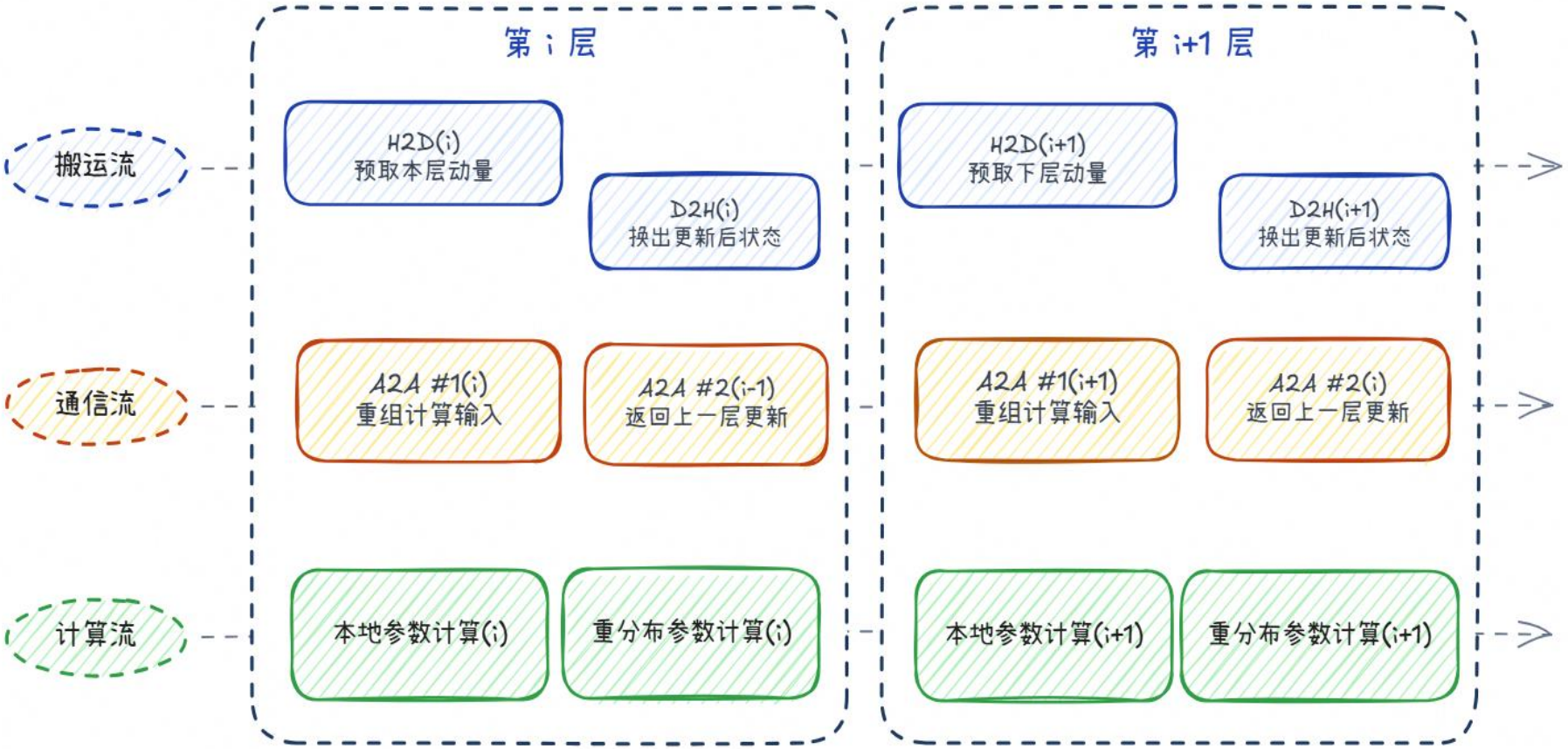
训得好

950超节点部署策略	Tilelang与自动融合	整网内存优化	1M长序列训练	训练图优化能力	高效分布式muon优化器	高可用能力

6. 高效分布式muon优化器：流水优化

流水优化：

- **动量预取：**进入每层前，NovaSwap提前进行H2D
- **AllToAllV预取：**当前实现先提交下一层的第一次 AllToAllV，再提交上一层的第二次 AllToAllV。



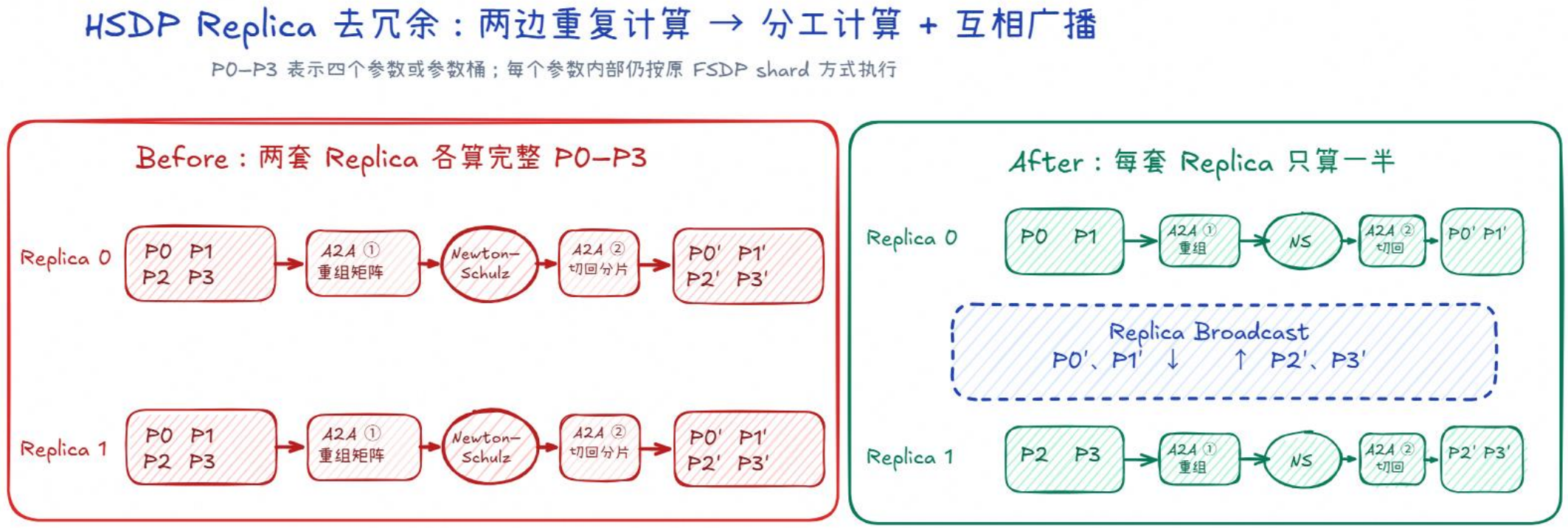
6. 高效分布式muon优化器：HSDP

HSDP 去冗余：计算一次，广播结果：

为每个参数选择一个 owner replica。只有owner rank执行这套计算和布局转换。

更新结束后，每个owner rank 分别把本地的新参数分片广播给对应副本。广播的是更新后的参数分片，不影响FSDP 的切分方式。

下一轮前向开始前，每套 replica 再次持有一致的参数。



https://gitcode.com/cann

CANN

训得好

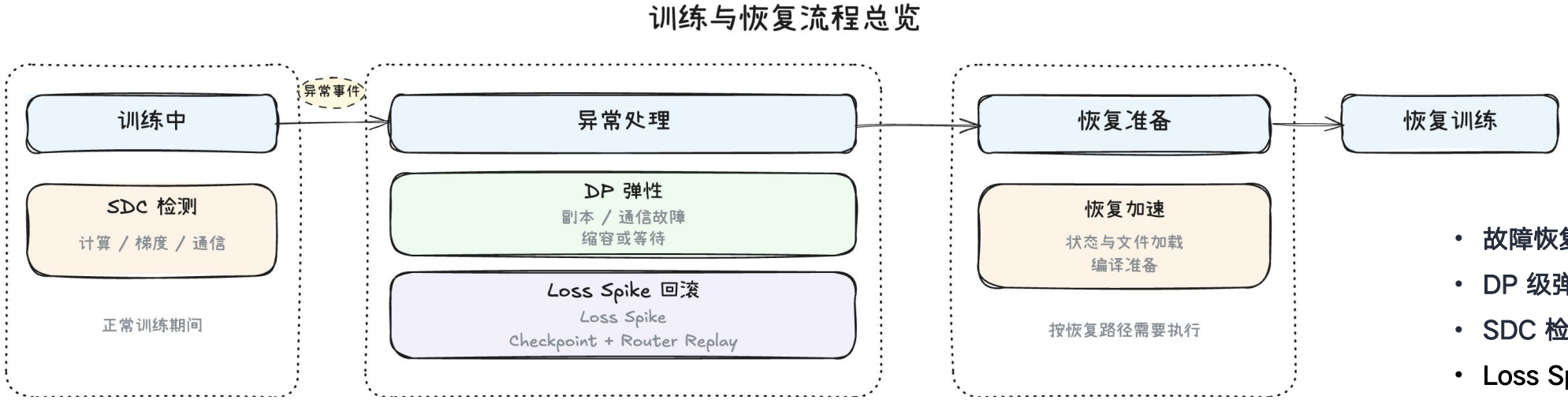
目录

Part 1 模型训得动

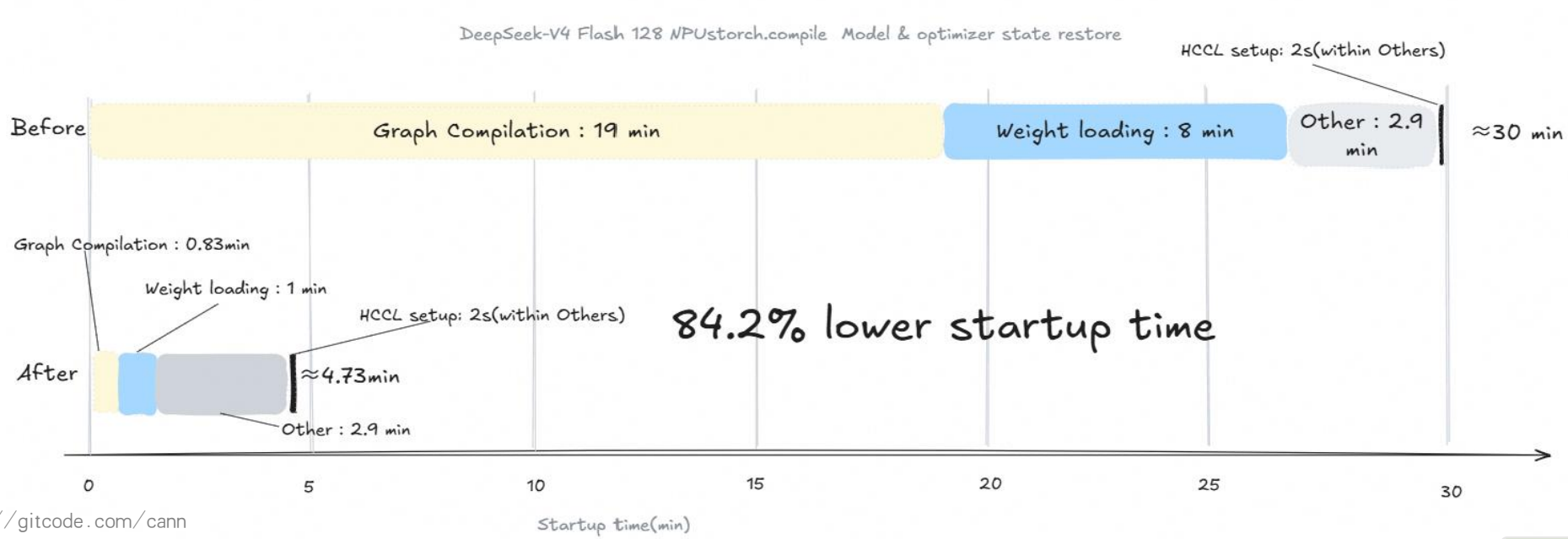
Part 2 模型训得好

Part 3 训练高可用

7. 面向千卡、万卡集群的预训练高可用能力



- 故障恢复加速
- DP 级弹性训练
- SDC 检测
- Loss Spike 回滚



https://gitcode.com/cann

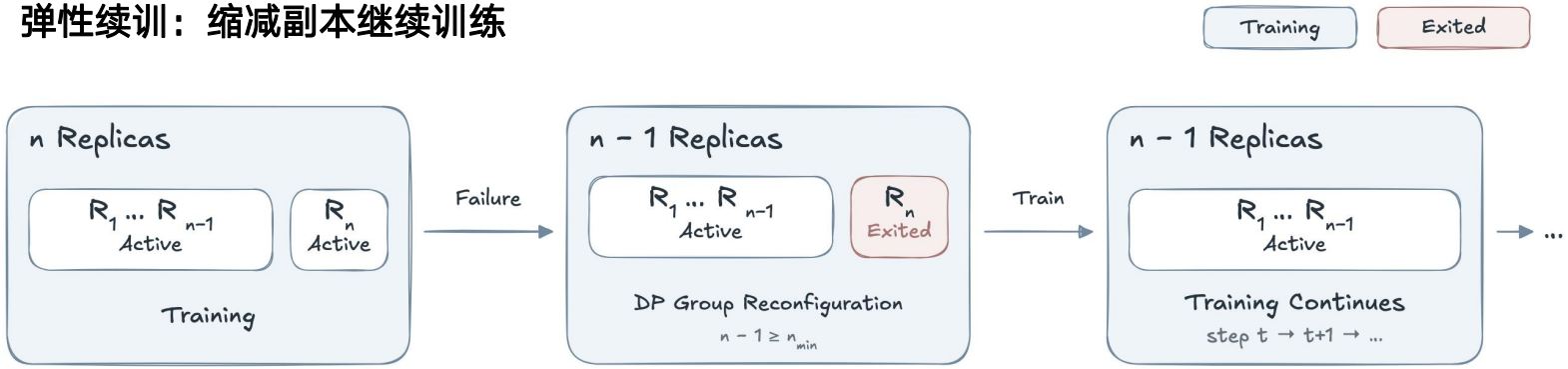
CANN

高可用

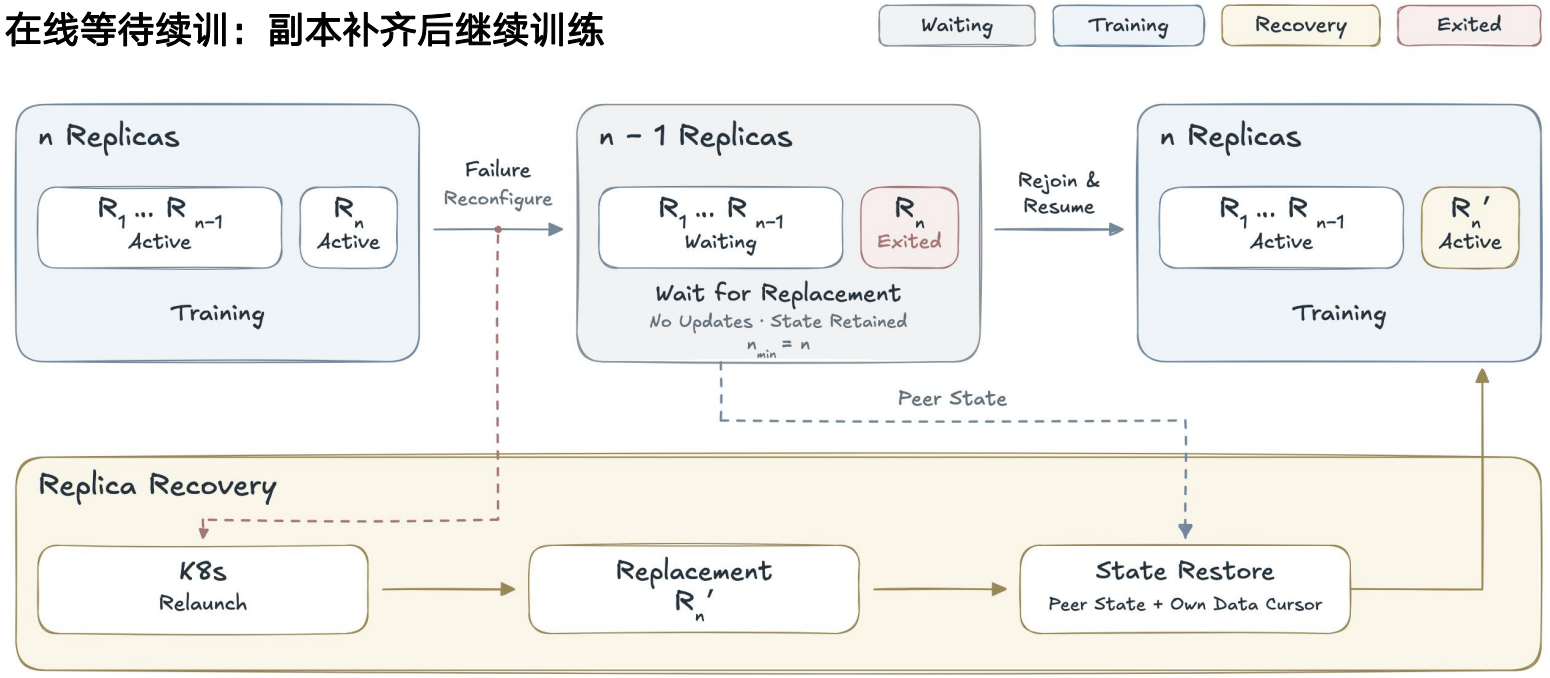
950超节点部署策略	Tilelang与自动融合	整网内存优化	1M长序列训练	训练图优化能力	高效分布式muon优化器	高可用能力

7. 面向千卡、万卡集群的预训练高可用能力: DP 级弹性训练

弹性续训：缩减副本继续训练



在线等待续训：副本补齐后继续训练



https://gitcode.com/cann

CANN

高可用

950超节点部署策略	Tilelang与自动融合	整网内存优化	1M长序列训练	训练图优化能力	高效分布式muon优化器	高可用能力

8. 总结

- 专家通信留在超节点高速域内，模型状态通过 FSDP、PP 和 Host 内存扩展容量；
- TileLang 和 AutoFuse 缩短新算法到高性能算子的距离；
- 细粒度重计算决定哪些激活值得保存，Offload 流水决定状态何时搬运；
- KVAllGather 和 ZigZag 让 1M 长序列既看得到完整历史，又保持负载均衡；
- GraphTrainer、EP overlap 和 CV 并行减少整步等待；
- FlexMuon 协同安排矩阵布局、状态驻留和计算时序；
- 高可用机制则保证发生故障后仍能回到一致、可信的训练状态。

欢迎体验和贡献

torchtitan-npu 开源讨论组

torchtitan-npu

欢迎关注：https://gitcode.com/cann/torchtitan-npu

欢迎广大开发者体验并参与贡献，如有疑问也可通过交流群联系我们！

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯